Assignment 03

Cortex-M Exception and Interrupt with PIT and RTC

Level	Fresher Embedded / MCU Fundamentals
Suggested duration	300 minutes
Submission	Report, source project, diagrams, and test evidence as applicable
Quality gate	Clear explanation, correct configuration, clean build, and reproducible result

1. Objective

Explain Cortex-M exception handling, configure interrupt control, and implement a software timer using a periodic timer interrupt. Extend the analysis to RTC configuration.

2. Part A - NVIC and exception analysis

- Draw the vector table organization and identify initial stack pointer, Reset handler, core exceptions, and external IRQ entries.
- Explain the NVIC role in enable/disable, pending, active state, priority, and interrupt selection.
- Explain the SCB role for system exception control, vector table location, fault status, and system control.
- Describe exception entry step by step: interrupt request, priority decision, context stacking, vector fetch, handler execution, return, and context restore.
- Explain optimization concepts such as short ISR design, deferred processing, tail-chaining, late arrival, and avoiding unnecessary work in handlers.

3. Part B - LPIT software timer

- Select a PIT/LPIT channel supported by the target and identify its clock source.
- Calculate the load value for a 1 ms interrupt from the actual timer clock.
- Configure module clock, timer mode, load value, interrupt enable, NVIC priority, and timer start.
- In the ISR, clear the correct flag and increment a millisecond tick.
- Implement a software-timer service supporting at least two timers with one-shot and periodic modes.
- Demonstrate one 500 ms event and one 1000 ms periodic event without busy-wait delays.

4. Suggested software interface

```
typedef void (*TimerCallback)(void);

void SoftTimer_Init(void);
bool SoftTimer_Start(uint8_t id, uint32_t timeout_ms, bool periodic, TimerCallback cb);
void SoftTimer_Stop(uint8_t id);
void SoftTimer_Process(void);
```

```
void PIT_IRQHandler(void);
```

~~5. Part C - RTC configuration analysis~~

- ~~Identify the RTC clock source, counter/time registers, status flags, alarm capability, and interrupt source.~~
- ~~Write an ordered RTC initialization sequence based on the selected manual.~~
- ~~Describe how to demonstrate current time and one alarm event. Actual RTC implementation is optional if the board example or hardware setup does not support it; the limitation must be documented.~~

6. Required test cases

- LPIT flag is cleared and no interrupt storm occurs.
- Tick interval is measured or observed against a reference.
- One-shot callback executes once.
- Periodic callback executes repeatedly.
- Stopping a timer prevents future callbacks.
- Invalid timer ID and NULL callback are handled.

Evaluation criteria

Category	Weight	Expected evidence
Technical understanding	30%	Accurate concepts, diagrams, and terminology.
Implementation / analysis	35%	Required configuration, code, or mapping is complete.
Testing and evidence	20%	Results are demonstrated and reproducible.
Documentation quality	15%	Clear structure, naming, references, and conclusions.